COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	09/08/26	**DATE SUBMITTED:**	09/09/26
NAME:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 **Sorting Algorithms Pt1**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for implementation of selection, bubble, insertion and merge sort.

Preparation Task:

- Create an array of elements with random values. The array must contain 100 elements that are not sorted. Use the created array for each sorting algorithm below. Show the output of the preparation task on table 7-1 in section 6.
- Create a header file for the implementation of the different sorting algorithms.
- Import this header file into your main.cpp.

A.1. Bubble Sort Technique

Using the bubble sort technique, sorting is done in passes or iteration. Thus at the end of each iteration, the heaviest element is placed at its proper place in the list. In other words, the largest element in the list bubbles up.

General Algorithm

```
template <typename T>
void bubbleSort(T arr[], size_t arrSize){
    //Step 1: For i = 0 to N-1 repeat Step 2
    for(int i = 0; i < arrSize; i++){
        //Step 2: For J = i + 1 to N - I repeat
        for(int j = i+1; j < arrSize; j++){
            //Step 3: if A[J] > A[i]
            if(arr[j]>arr[i]){
                //Swap A[J] and A[i]
                std::swap(arr[j], arr[i]);
            }
            //[End of Inner for loop]
        }
        //[End if Outer for loop]
    }
    //Step 4: Exit
}
```

Include a screenshot of your code and output console along with your observations in table 7-2.

A.2. Selection Sort Algorithm

Selection sort is quite a straightforward sorting technique as the technique only involves finding the smallest element in every pass and placing it in the correct position. Selection sort works efficiently when the list to be sorted is of small size but its performance is affected badly as the list to be sorted grows in size.
Hence we can say that selection sort is not advisable for larger lists of data.

```
template <typename T> int Routine_Smallest(T A[], int K, const int arrSize);

//General Algorithm
//Selection Sort (Array, Size of Array)
template <typename T>
void selectionSort(T arr[], const int N){
    int POS, temp, pass=0;
    //Step 1: Repeat Steps 2 and 3 for K = 1 to N-1
    for(int i = 0; i < N; i++){
        //Step 2: Call routine smallest(A, K, N,POS)
        POS = Routine_Smallest(arr, i, N);
        temp = arr[i];
        //Step 3: Swap A[K] with A [POS]
        arr[i] = arr[POS];
        arr[POS] = temp;
        //Count
        pass++;
    }
    //[End of loop]
    //Step 4: EXIT
}

//Routine smallest (Array, Current Position, Size of Array)
template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize){
    int position, j;
    //Step 1: [initialize] set smallestElem = A[K]
    T smallestElem = A[K];
    //Step 2: [initialize] set POS = K
    position = K;
    //Step 3: for J = K+1 to N -1,repeat
    for(int J=K+1; J < arrSize; J++){
        if(A[J] < smallestElem){
            smallestElem = A[J];
```

```
                position = J;
            }
        }
        //Step 4: return POS
        return position;
    }
```

Include a screenshot of your code and output console along with your observations in table 7-3.

A.3. Insertion Sort Algorithm

In the insertion sort technique, we start from the second element and compare it with the first element and put it in a proper place. Then we perform this process for the subsequent elements. We compare each element with all its previous elements and put or insert the element in its proper position. Insertion sort technique is more feasible for arrays with a smaller number of elements. It is also useful for sorting linked lists. Linked lists have a pointer to the next element (in case of a singly linked list) and a pointer to the previous element as well (in case of a doubly linked list). Hence it becomes easier to implement insertion sort for a linked list.

General Algorithm

```
//General Algorithm
//Insertion Sort
template <typename T>
void insertionSort(T arr[], const int N){
    int K = 0, J, temp;
    //Step 1: Repeat Steps 2 to 5 for K = 1 to N-1
    while(K < N){
        //Step 2: set temp = A[K]
        temp = arr[K];
        //Step 3: set J = K - 1
        J = K-1;
        //Step 4: Repeat while temp <=A[J]
        while(temp <= arr[J]){
            //set A[J + 1] = A[J]
            arr[J+1] = arr[J];
            //set J = J - 1
            J--;
            //[end of inner loop]
        }
        //Step 5: set A[J + 1] = temp
        arr[J+1] = temp;
        //[end of loop]
        K++;
    }
    //Step 6: exit
}
```

Include a screenshot of your code and output console along with your observations in table 7-4.

ILO A OUTPUTS:

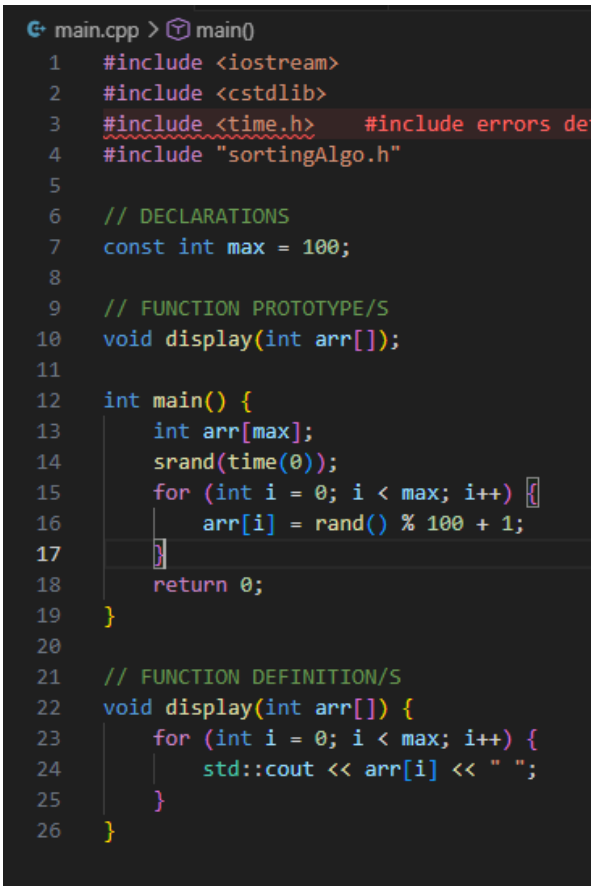
Code + Console Screenshot	
Observations	The following code works by generating an array with 100 randomized elements within it using the random and time functions as a seed to generate said elements. Additionally, a function called **display()** is prototyped and defined which basically travels through the entire array and prints out the elements within it. Given that everything is only executed once, the time complexity is therefore **O(n)**.

Table 7-1. Array of Values for Sort Algorithm Testing

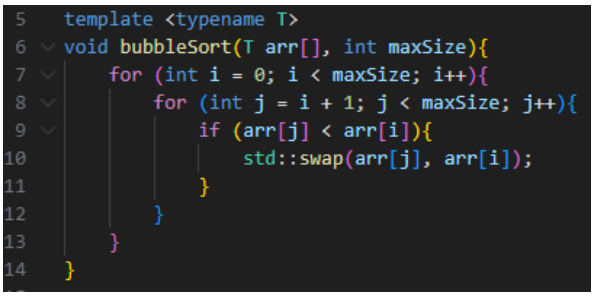
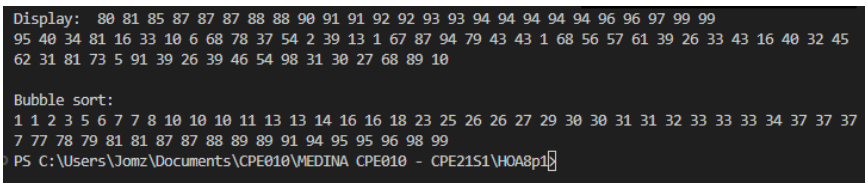
Code + Console Screenshot	 
Observations	This algorithm works by making use of bubble sort which, in this case, compares elements that are adjacent to each repeatedly and swaps them whenever they are in incorrect order. By changing the symbol between comparing arr[j] and arr[i] to a greater than or less than symbol, you can change how the array is sorted by descending or ascending order respectively. Additionally, since we have nested **for loops** for this sorting technique, its worst time complexity would probably be **O(n^2)** with the best being that the case that the array is already sorted having a time complexity of **O(n)**

Table 7-2. Bubble Sort Technique

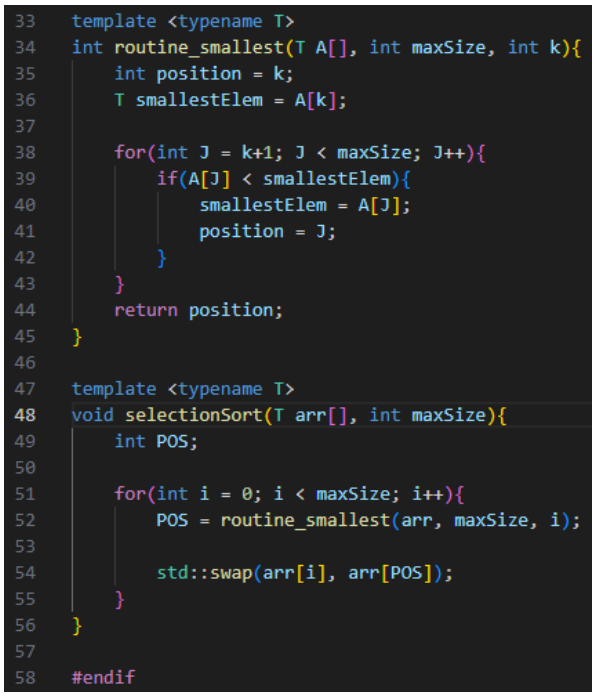
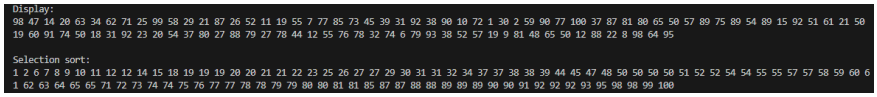
Code + Console Screenshot	 
Observations	The code above shows a selection sort algorithm being used where the remaining unsorted portion of the array is observed in order to find the smallest element. After, it swaps the value at the current position with the smallest element which builds the sorted part continuously from the left to the right. The time complexity is **O(n^2)** for both best and worst cases due to nested **for loops** called inside the selection sort where its for loop uses the function '**routine_smallest()**' which also uses a **for loop** inside it.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	
Observations	The code above makes use of the insertion sort algorithm where it starts comparing the second element to the elements before it where the larger values that are found later on are shifted repeatedly towards the left until the current value is inserted into the correct position. The best case time complexity for this algorithm is **O(n)** for the case that the array is already sorted with the worst case being **O(n^2)** due to the nested **while loops**.

Table 7-4. Insertion Sort Algorithm

2. SUPPLEMENTARY OUTPUT

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

Deliverables:

- Pseudocode of Algorithm
- Screenshot of Algorithm Code
- Output Testing

Pseudocode of Algorithm

```
START
DECLARE and INITIALIZE an array 'A' sized 100 with a seeded random number generator limited from 1-5.
DECLARE and INITIALIZE to 0, 'voteCount' with 5 elements

FOR each position in array 'A'
        generate random numbers with a range of 1-5
        store numbers into array 'A'

DISPLAY unsorted array, 'A'
CALL insertion sort on array 'A'
DISPLAY sorted array 'A'

FOR each vote in sorted 'A'
        Increment voteCount[vote - 1]

highest = first vote count of array, 'voteCount'

FOR each candidate
        PRINT candidate name and vote count
        IF candidate vote count > highest
                highest = array, candidate's vote count
```

PRINT “Winning Candidate:”
FOR each candidate
IF array, candidate’s vote count = highest
PRINT candidate with votes

END

Screenshot of Algorithm Code

```cpp
#include <iostream>
#include <cstdlib>
#include "sortingAlgo.h"
#include <string>

// DECLARATIONS
const int max = 100;

// FUNCTION PROTOTYPE/S
void display(int arr[]);

int main() {
    int A[max];
    srand(67);
    for(int i = 0; i < max; i++) {
        A[i] = rand() % 5 + 1;
    }

    int voteCount[5] = {0, 0, 0, 0, 0};
    std::string candidates[5] = {"Bo Dalton Capistrano", "Cornelius Raymon Agustin", "Deja Jayla Banaga", "Lalla Brielle Yabut", "Franklin Relano Castro"};

    std::cout<<"Unsorted Votes: \n";
    display(A);

    std::cout<<"\n\nSorted Votes: \n";
    insertionSort(A, max);
    display(A);

    for(int i = 0; i < max; i++){
        voteCount[A[i]-1]++;
    }

    std::cout<<"\n\nVoting Results:\n";
    int highest = voteCount[0];
    for(int i=0; i < 5; i++){
        std::cout<< candidates[i] << ": " << voteCount[i] << " votes\n";

        if(voteCount[i] > highest) {
            highest = voteCount[i];
        }
    }

    std::cout << "\nWinning Candidate: \n";
    for(int i = 0; i < 5; i++){
        if (voteCount[i] == highest){
            std::cout << candidates[i] << " with " << highest << " votes takes the lead!";
        }
    }
```

```cpp
    return 0;
}

// FUNCTION DEFINITION/S
void display(int arr[]) {
    for (int i = 0; i < max; i++) {
        std::cout << arr[i] << " ";
    }
}
```

Output Testing

```
Unsorted Votes:
3 2 2 3 4 5 2 2 1 3 2 3 3 2 5 4 1 2 2 5 2 4 2 5 3 1 3 3 5 2 2 3 2 4 5 1 1 5 4 5 2 3 5 4 3 2 4 5 2 2 5 4 5 3 5 1 4 1 3 3 2 1 1 2
5 5 3 1 4 3 4 5 4 2 1 5 3 1 4 3 5 1 3 3 5 1 3 5 5 3 3 3 3 3 1 5 1 2 3 5

Sorted Votes:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Voting Results:
Bo Dalton Capistrano: 16 votes
Cornelius Raymon Agustin: 21 votes
Deja Jayla Banaga: 27 votes
Lalla Brielle Yabut: 13 votes
Franklin Relano Castro: 23 votes

Winning Candidate:
Deja Jayla Banaga with 27 votes takes the lead!
```

Output Console Showing Array	Manual Count	Count Result of Algorithm
Bo Dalton Capistrano	16	16
Cornelius Raymon Agustin	21	21
Deja Jayla Banaga	27	27
Lalla Brielle Yabut	13	13
Franklin Relano Castro	23	23

Sorting Method Justification:

The sorting method that I had used for this activity is that of the insertion sort algorithm. Given that the array used to generate and hold the votes, 'A' is only up to a hundred elements, the sample space is small enough to be handled efficiently by the sorting method where the identical votes are placed beside each other making readability easier in manual checking.

Question: Was your developed vote counting algorithm effective? Why or why not?

Yes, given an unsorted, seeded randomly generated array, I believe that my developed vote counting algorithm is effective because it correctly does all the things that are needed like with generating, sorting, as well as counting all of the 100 votes for the candidates. Additionally, the manual and algorithm counts match each other and the highest voted candidate is correctly chosen. Finally, the time complexity sits at a solid **O(n^2)** due to insertion sort being the most time consuming operation here, but other than that, the rest of the program runs only once in **O(n)** and **O(1)**.

Conclusion:

In this activity, I learned a lot about the what and how bubble, selection, and insertion sort works at organizing data within data structures, specifically, arrays using loops in the main procedure. Additionally, I also looked into their time complexities as well as applying a sorting technique of my choosing in a practical problem such as with counting votes in the supplementary activity. I believe I've done well in this activity and understand what each sorting algorithm does. I'm just having a little trouble with exactly how the code works and that I could improve by really honing it in my mind completely, but other than that, it's all clear to me. Overall, the activity has shown me that it is important to choose the appropriate algorithm for certain cases.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.